ایده‌ها و الگوهای حل مسئله + حل کامل میان‌ترم‌های ۹۸ تا ۱۴۰۲

ساختار و زبان کامپیوتر — دکتر سربازی‌آزاد — از ساده تا پیشرفته، با کد. همهٔ کدها روی شبیه‌ساز همان ماشین اجرا و آزموده شده‌اند.

سطح ۱ پایه | سطح ۲ آرایه | سطح ۳ حساب و بیت | سطح ۴ مرتب‌سازی | سطح ۵ ترفند و خودتغییرده

۱. نقشهٔ امتحان: میان‌ترم‌های ۹۸ تا ۱۴۰۲ در یک نگاه ۱
۲. دستور پخت سؤال ۱: طول ثبات‌ها ۱
۳. دستور پخت سؤال ۴: ترجمه به کد ماشین ۱
۴. دستور پخت سؤال ۳: «این برنامه چه می‌کند؟» ۲
۵. الگوهای سطح ۱: پایه ۲
۶. الگوهای سطح ۲: آرایه ۳
۷. الگوهای سطح ۳: حساب و بیت ۴
۸. الگوهای سطح ۴: مرتب‌سازی ۵
۹. الگوهای سطح ۵: کد خودتغییرده و ترفندهای پیشرفته ۵
۱۰. ممیز شناور در برنامه‌ها ۶
۱۱. ایده‌های اسلایدها و جزوه (مرور سریع) ۶
۱۲. حل کامل میان‌ترم ۱۴۰۲ ..... ۶
۱۳. حل کامل میان‌ترم ۱۴۰۱ ..... ۷
۱۴. حل کامل میان‌ترم ۱۴۰۰ ..... ۸
۱۵. حل کامل میان‌ترم ۱۳۹۹ ..... ۹
۱۶. حل کامل میان‌ترم ۱۳۹۸ ..... ۹

۱ نقشهٔ امتحان: میان‌ترم‌های ۹۸ تا ۱۴۰۲ در یک نگاه

الگوی ثابت هر سال (۹۰ دقیقه، ۱۰ نمره)

هر سال یک **ماشین فرضی تازه** با جدول کد عمل داده می‌شود و **همیشه همین چهار سؤال** می‌آید:

سؤال	موضوع	نمره	کجای این برگه
۱	طول همهٔ ثبات‌های ماشین	۱	بخش ۲
۲	نوشتن برنامه (مرتب‌سازی، فیبوناچی، سری، جمع متناوب، تکمیل برنامه)	۳ تا ۴	بخش‌های ۵ تا ۱۰ و حل‌ها
۳	«این برنامه چه می‌کند؟» با کد فریبنده	۳	بخش ۴ و ۹
۴	ترجمهٔ برنامهٔ سؤال ۳ به کد ماشین	۲ تا ۳	بخش ۳

سال	حافظه / واحد / کلمه	ثبات / پرچم	سؤال ۲	ترفند سؤال ۳	جواب سؤال ۳
۱۴۰۲	2^{24} × 6b / word 24 (4u)	PSR: z s	پرکردن A_i با جمع متناوب پیشوندی	برچسب‌های here±k، ماسک دوبرابرشونده، `sub #FFFFFFh` = 1+	s = تعداد بیت‌های ۱ عدد c
۱۴۰۱	2^{21} × 7b / word 21 (3u)	PSR: z s v	تکمیل مرتب‌سازی Gnome با جابه‌جایی XOR	XOR روی خود دستور: `jmp next` ⇄ `add`	a = -10 (یک دور در میان کم می‌کند)
۱۴۰۰	2^{14} × 16b, unit 4b	—	جمع ۱۰۰ جملهٔ فیبوناچی	dw در وسط کد = ۴ دستور؛ کد خودتغییرده	A[0..99] = فیبوناچی
۱۳۹۹	2^{24} × 6b / word 24 (4u)	R0..R31	مرتب‌سازی صعودی ۱۰۰ کلمه	`dw 41006h` = دستور؛ پرش به `loop-6`؛ #100 بر حسب واحد	وارونه کردن ۲۶ کلمهٔ اول
۱۳۹۸	2^{29} × 32b, unit 4b	R0..R15 (R0=0)	e^x با دقت 2^{-10}	اعداد ممیز شناور که در واقع دستورند؛ اجرا از org 0	شمارش NaN و غیرنرمال‌ها

روند پیشنهادی سر جلسه

1. جدول ماشین را بخوان و اول **طول هر قالب بر حسب واحد** را کنار جدول بنویس (۱ دقیقه، به همهٔ سؤال‌ها کمک می‌کند).
2. سؤال ۱ را فوراً بنویس (بخش ۲).
3. سؤال ۳: آدرس همهٔ خط‌ها را حساب کن، سپس کد را **واقعاً** اجرا کن (جدول ردگیری) — بخش ۴.
4. سؤال ۴ با همان آدرس‌ها تقریباً آماده است؛ فقط فیلدها را کنار هم بگذار.
5. سؤال ۲ را از الگوهای آماده بساز و با یک ورودی کوچک دستی امتحان کن.

چیزهایی که هر سال عوض می‌شود

- **طول واحد آدرس‌پذیر** (۴، ۶، ۷ بیت) ⇒ گام اشاره‌گر = کلمه/واحد (۸، ۴، ۳، ۴).
- **شرط پرش‌ها:** `jpos` گاهی ≤ 0 (۱۳۹۹) و گاهی < 0 (۱۴۰۱: `svz` = 0).
- **پرچم‌ها:** فقط دستورهای حسابی/منطقی PSR را عوض می‌کنند (نه `mov`).
- **بازگشت به OS:** `jmp R31`، `jmp (FFFEh)`، `jmp (1FFFFDh)`، `je R0,R0,OS`، `jz OS`.
- **بیت‌های حالت آدرس‌دهی** (i, d, x, y, z) جای متفاوتی در قالب دارند.

۲ دستور پخت سؤال ۱: طول ثبات‌ها

قاعده

- **MAR = PC** = تعداد بیت آدرس = $\log_2$(واحدها#). اگر حافظه بر حسب کلمه داده شد، اول به واحد تبدیل کن: #units = #words × (w/u).
- **MBR** = طول کلمه (پهنای دسترسی).
- **IR** = طول **بلندترین** قالب دستور (بیت).
- **ثبات‌های همه‌منظوره / ACC** = طول کلمه؛ **PSR** = تعداد پرچم‌ها؛ **SP** = $\log_2$(اندازهٔ پشته) یا طول آدرس.

جواب پنج سال

Year	#units	MAR	PC	MBR	IR	Others
1402	2^{24}	24	24	24	54 (F I)	PSR 2
1401	2^{21}	21	21	21	49 (F I)	PSR 3
1400	$2^{14}\times4=2^{16}$	16	16	16	52 (F III)	—
1399	2^{24}	24	24	24	36 (F III)	R0..R31: 24
1398	$2^{29}\times8=2^{32}$	32	32	32	48 (F III)	R0..R15: 32

۳ دستور پخت سؤال ۴: ترجمه به کد ماشین

گام‌ها

1. **طول قالب‌ها بر حسب واحد:** بیت‌های قالب ÷ بیت واحد (باید بخش‌پذیر باشد).
2. **شمارندهٔ مکان:** از `org`؛ هر خط LC += طول؛ `dw` = تعداد مقدار × واحدهای کلمه؛ برچسب‌ها را در جدول نمادها بنویس.
3. **عبارت‌های آدرس** مثل `A+400`، `c-3`، `loop-6` و `here+14` همگی بر حسب **واحد** حساب می‌شوند، نه کلمه.
4. برای هر دستور: کد عمل ‖ بیت‌های حالت ‖ فیلدها؛ اعداد منفی به مکمل دو با **طول همان فیلد** (`-2` در ۲۱ بیت = 1FFFFEh).
5. نمایش: واحد ۴ بیتی ⇐ هر رقم هگز یک واحد. واحد ۶ بیتی ⇐ هر واحد = ۲ رقم **اکتال**. واحد ۷ بیتی ⇐ دودویی فیلدها (یا هگز هر واحد).
6. `dw ?` را با ???? یا «نامعلوم» بنویس؛ برچسب بیرونی (OS) را نمادین بگذار.

طول قالب‌ها (حفظ نکن، حساب کن)

Year	unit	word	F I	F II	F III	F V
1402	6	4u	54b = 9u	30b = 5u	—	—
1401	7	3u	49b = 7u	28b = 4u	—	—
1400	4	4u	36b = 9u	20b = 5u	52b = 13u	—
1399	6	4u	18b = 3u	12b = 2u	36b = 6u	30b = 5u
1398	4	8u	40b = 10u	12b = 3u	48b = 12u	—

تشخیص قالب از بیت‌های اول (برای سؤال ۳): ۱۳۹۹: …1=۱۳۹۸. III=۱۱، 011=III، 010=II، 00=I، V: نیبل اول F=III=0، ۱، بقیه=۱۴۰۰: II. کد عمل 000–010=III، 101–111=III، 011–100=II، I
۱۴۰۱/۱۴۰۲: اگر دو بیت حالت عملوند دوم 11 باشد ⇐ قالب II.

۴ دستور پخت سؤال ۳: «این برنامه چه می‌کند؟»

چک‌لیست رمزگشایی (به ترتیب)

1. **آدرس هر خط را حساب کن** (مثل سؤال ۴). بدون آدرس دقیق نمی‌شود فهمید `loop-6` یا `here+14` کجاست.
2. **نقطهٔ شروع:** اجرا از اولین `org` شروع می‌شود؛ اگر آنجا `dw` است، آن عدد را **دستور** فرض کن (۱۳۹۸: `dw -1.0e+109` = `je R0,R0,330h`).
3. **هر `dw` وسط کد:** بیت‌هایش را بنویس و از روی بیت‌های اول قالب را تشخیص بده (۱۳۹۹: `dw 41006h` = `add R1,R0`).
4. **پرش به وسط دستور** (`loop-6`): از همان واحد به بعد، بیت‌ها را از نو رمزگشایی کن؛ ممکن است دستور بعدی (مثل `mov R1,#807004h`) هرگز آن‌طور که نوشته شده اجرا نشود.
5. **عملوندی که به کد اشاره می‌کند** (`loop+1`، `Arr+10`، `s-4`): ببین کدام فیلد کدام دستور است ⇐ کد خودتغییرده.
6. **عملوندهای هم‌پوشان:** `c-3` ممکن است همان `a` باشد (`xor a,a` = صفر)، `s-4` همان `c+4`.
7. **«#عدد» بر حسب واحد است:** `mov R1,#100` با کلمهٔ ۴ واحدی یعنی ۲۵ کلمه.
8. **جدول ردگیری** با ۲ تا ۳ دور کامل بکش و الگو را پیدا کن؛ سپس شرط پایان (شمارنده، سرریز ماسک، برخورد اشاره‌گرها) را بررسی کن.
9. جواب را **به زبان مسئله** بنویس (مثلاً «تعداد بیت‌های ۱ را می‌شمارد») و جزئیات حاشیه‌ای (عنصری که بررسی نمی‌شود، مقدار نهایی متغیرها) را هم بگو.

فرمول تغییر یک فیلد با add روی کلمه

کلمه‌ای که از واحد k دستور شروع می‌شود، بیت‌های $[k{\cdot}u,\ k{\cdot}u + w)$ دستور را می‌پوشاند. اگر کم‌ارزش‌ترین بیت فیلد، e بیت بالاتر از کم‌ارزش‌ترین بیت این کلمه باشد، برای افزودن d به فیلد باید $d{\cdot}2^e$ به کلمه اضافه کرد.

- ۱۴۰۲: `add loop+5,#4` ⇐ فیلد addr2 دقیقاً واحدهای ۵ تا ۸ (e = 0) ⇒ آدرس +۴.
- ۱۴۰۰: `add Arr+10,#8` ⇐ کلمه = بیت‌های ۴..۱۹ دستور، addr1 = بیت‌های ۳..۱۸ ⇒ e = ۸ ⇒ یعنی آدرس +۴.
- ۱۴۰۰: `add Arr+14,#4` ⇐ addr2 = بیت‌های ۲۰..۳۵ = دقیقاً واحدهای ۵..۸ ⇒ +۴.

جدول ترفندهای دیده‌شده

داده‌ای که دستور است	۹۸، ۹۹، ۱۴۰۰
پرش به وسط دستور	۹۹ (loop-6)، (here±k) ۱۴۰۲
تغییر فیلد آدرس برای پیمایش آرایه	۱۴۰۰، اسلاید ماشین ۳
XOR روی دستور برای تبدیل آن	۱۴۰۱
ماسک ۱،۲،۴،... تا سرریز	۱۴۰۲
دو اشاره‌گر که به هم می‌رسند	۹۹
R0 همیشه صفر (`je R0,R0` ، `x(R0)`)	۹۸
تفریق با -۱ = افزایش	۱۴۰۲
برچسب‌های بی‌استفاده برای گمراه کردن	۱۴۰۱ (arr، k)

۵ الگوهای سطح ۱: پایه

نمادگذاری کدهای بخش‌های ۵ تا ۹: **ماشین میان‌ترم ۱۴۰۲** (دوآدرسهٔ حافظه-حافظه، کلمه = ۴ واحد، پرچم z و s؛ `#` بی‌واسطه، `(x)` غیرمستقیم). در ماشین‌های دیگر فقط گام اشاره‌گر و نام پرش‌ها عوض می‌شود. `jmp OS` یعنی بازگشت به سیستم‌عامل.

۱ صفر کردن، ±۱ و تست مقدار

`xor x,x` یا `mov x,#0`؛ `sub x,#FFFFFFh` = x+1 (۱۴۰۲)؛ یا `add y,#0` فقط برای به‌روز کردن پرچم‌ها. `mov t,#0 / or t,y`

```
        xor  x, x          ; x = 0 (or mov x,#0)
        sub  x, #FFFFFFh   ; x = x - (-1) = x + 1
        add  x, #FFFFFFh   ; x = x + (-1) = x - 1
        add  y, #0         ; PSR ← y, y unchanged
        jneg yneg
        jmp  OS
yneg:   mov  flag, #1
        jmp  OS
x:      dw   77
y:      dw   -5
flag:   dw   0
```

۱ حلقهٔ شمارشی

شمارنده را از N کم کن و `jnz` بزن (پرچم از `sub`). معادل‌ها: `loop r,addr` (۹۹)، `loop-` / `loop+` (۹۸)، `dec n` / `jne n,#0,L` (۱۴۰۰).

```
        mov  n, #10        ; repeat 10 times
        mov  sum, #0
loop:   add  sum, n        ; body: sum = 10+9+...+1
        sub  n, #1
        jnz  loop
        jmp  OS
n:      dw   ?
sum:    dw   ?
```

۱۴۰۰ ۱۴۰۲ همهٔ اسلایدها

۱ جابه‌جایی با XOR (بدون متغیر کمکی)

a⊕b⊕b = a. شرط: a و b **دو خانهٔ متفاوت** باشند (اگر یک خانه باشند، صفر می‌شود!).

```
        xor  a, b          ; a = a^b
        xor  b, a          ; b = b^(a^b) = a
        xor  a, b          ; a = (a^b)^a = b
        jmp  OS
a:      dw   1234h
b:      dw   0ABCDEh
```

۱۴۰۱ اسلاید ماشین ۲

۱ جابه‌جایی با موقت / با اشاره‌گر

```
        mov  t, a
        mov  a, b
        mov  b, t
        jmp  OS
a:      dw   5
b:      dw   9
t:      dw   ?
```

```
        xor  (p), (q)      ; swap *p and *q
        xor  (q), (p)
        xor  (p), (q)
        jmp  OS
p:      dw   x
q:      dw   y
x:      dw   7
y:      dw   3
```

جمع/تفریق: a=a+b; b=a-b; a=a-b (در ماشین دوآدرسه یک منفی‌سازی لازم دارد). ۱۳۹۹ دستور `swap (R1),(R2)` دارد.

۱ پرش بدون شرط و بازگشت

Machine	Always jump	Return to OS
1402	jmp L	jz/jmp OS
1401	jmp L	jmp (1FFFFDh)
1400	je x,x… / jmp (p)	jmp (FFFEh)
1399	jmp+ Rk,L / jnz R≠0,L	jmp R31
1398	je R0,R0,L	je R0,R0,OS
M3 slide	jnz one,L	jnz one,OS

`call` آدرس بازگشت را در ثبات (۹۹: `call r,addr`) یا خانهٔ ثابت (۱۴۰۰: FFFEh، ۱۴۰۱: 1FFFFDh) می‌گذارد و بازگشت با پرش غیرمستقیم است.

۶ الگوهای سطح ۲: آرایه

۲ جمع آرایه با اشاره‌گر

اشاره‌گر `p` با `#A` شروع و هر دور یک کلمه (اینجا ۴ واحد) جلو می‌رود.

```
        mov  p, #A          ; p → A[0]
        mov  n, #100
        mov  sum, #0
loop:   add  sum, (p)       ; sum += A[i]
        add  p, #4          ; next word = 4 units
        sub  n, #1
        jnz  loop
        jmp  OS
p:      dw   ?
n:      dw   ?
sum:    dw   ?
A:      dw   100 dup(?)
```

۲ بیشینه

مقایسه = تفریق در متغیر موقت و پرش روی علامت.

```
        mov  p, #A
        mov  mx, (p)        ; mx = A[0]
        mov  n, #99
loop:   add  p, #4
        mov  t, (p)
        sub  t, mx          ; t = A[i] - mx
        jneg skip
        mov  mx, (p)        ; new maximum
skip:   sub  n, #1
        jnz  loop
        jmp  OS
p:      dw   ?
n:      dw   ?
mx:     dw   ?
t:      dw   ?
A:      dw   100 dup(?)
```

۲ شمارش عناصر منفی

ترفند «تست»: `mov t,#0 / or t,(p)` مقدار را می‌آورد و پرچم‌ها را هم تنظیم می‌کند.

```
        mov  p, #A
        mov  n, #100
        mov  cnt, #0
loop:   mov  t, #0
        or   t, (p)         ; t = A[i], PSR ← A[i]
        jneg neg
        jmp  next
neg:    add  cnt, #1
next:   add  p, #4
        sub  n, #1
        jnz  loop
        jmp  OS
p:      dw   ?
n:      dw   ?
cnt:    dw   ?
t:      dw   ?
A:      dw   100 dup(?)
```

۲ وارونه کردن با دو اشاره‌گر

p از اول و q از آخر؛ جابه‌جایی و نزدیک شدن تا وقتی p > q. (همان ایدهٔ سؤال ۳ سال ۹۹.)

```
        mov  p, #A          ; p → first
        mov  q, #A+396      ; q → last (99 x 4)
loop:   mov  t, q
        sub  t, p           ; t = q - p
        jneg done           ; pointers crossed
        jz   done           ; or met
        xor  (p), (q)       ; swap A[i], A[j]
        xor  (q), (p)
        xor  (p), (q)
        add  p, #4
        sub  q, #4
        jmp  loop
done:   jmp  OS
p:      dw   ?
q:      dw   ?
t:      dw   ?
A:      dw   100 dup(?)
```

۱۳۹۹

۲ جمع پیشوندی

```
        mov  s, #0          ; A[i] = A[0]+...+A[i]
        mov  p, #A
        mov  n, #100
loop:   add  s, (p)
        mov  (p), s
        add  p, #4
        sub  n, #1
        jnz  loop
        jmp  OS
s:      dw   ?
p:      dw   ?
n:      dw   ?
A:      dw   100 dup(?)
```

۲ جمع متناوب پیشوندی $A_i = \Sigma_{j\le i}(-1)^j A_j$

علامت را با **باز کردن حلقه به اندازهٔ ۲** مدیریت کن: یک بار add (j زوج) و یک بار sub (j فرد). مقدار قدیمی A_j قبل از بازنویسی خوانده می‌شود.

```
        mov  s, #0          ; s = running sum
        mov  p, #A
        mov  n, #50         ; two elements per pass
loop:   add  s, (p)         ; even j: +A[j]
        mov  (p), s
        add  p, #4
        sub  s, (p)         ; odd j:  -A[j]
        mov  (p), s
        add  p, #4
        sub  n, #1
        jnz  loop
        jmp  OS
s:      dw   ?
p:      dw   ?
n:      dw   ?
A:      dw   100 dup(?)
```

۱۴۰۲ سؤال ۲

۲ مجموع n جملهٔ فیبوناچی

ترفند «دو جمله در هر دور»: `a+=b; b+=a` بدون متغیر موقت. اتحاد: $\Sigma F_{0..n-1} = F_{n+1} - 1$.
حواست به سرریز باشد (جواب واقعی برای ۱۰۰ جمله خیلی بزرگ است).

```
        mov  a, #0          ; F0
        mov  b, #1          ; F1
        mov  sum, #1        ; F0 + F1
        mov  n, #49         ; 2 terms per pass
loop:   add  a, b           ; a = F2, F4, ...
        add  sum, a
        add  b, a           ; b = F3, F5, ...
        add  sum, b
        sub  n, #1
        jnz  loop
        jmp  OS
a:      dw   ?
b:      dw   ?
sum:    dw   ?
n:      dw   ?
```

۱۴۰۰ سؤال ۲ | اسلاید ماشین‌های ۲، ۳، ۶

۲ ساختن فیبوناچی در آرایه

A[i] = A[i-1] + A[i-2] با یک اشاره‌گر که عقب و جلو می‌رود.

```
        mov  p, #A
        mov  (p), #0        ; A[0] = 0
        add  p, #4
        mov  (p), #1        ; A[1] = 1
        mov  n, #98
loop:   mov  t, (p)         ; t = A[i-1]
        sub  p, #4
        add  t, (p)         ; t = A[i-1] + A[i-2]
        add  p, #8
        mov  (p), t         ; A[i] = t
        sub  n, #1
        jnz  loop
        jmp  OS
p:      dw   ?
n:      dw   ?
t:      dw   ?
A:      dw   100 dup(?)
```

۱۴۰۰ سؤال ۳ | اسلاید تک‌آدرسه

۷ الگوهای سطح ۳: حساب و بیت

۳ ضرب با جمع‌های متوالی

```
        mov  r, #0          ; r = a*b  (b ≥ 0)
        mov  t, #0
        or   t, b           ; test b
        jz   done
loop:   add  r, a
        sub  b, #1
        jnz  loop
done:   jmp  OS
a:      dw   -7
b:      dw   12
r:      dw   ?
t:      dw   ?
```

۳ ضرب با شیفت و جمع

شیفت به چپ = `add a,a` . شیفت به راست نداریم، پس به‌جای شیفت b، ماسک را دوبرابر می‌کنیم؛ حلقه با سرریز ماسک (بعد از w دور) تمام می‌شود.

```
        mov  r, #0          ; r = a*b, shift-and-add
        mov  mask, #1
loop:   mov  t, b
        and  t, mask        ; is this bit of b set?
        jz   skip
        add  r, a           ; r += a * 2^k
skip:   add  a, a           ; a ⟸ 1
        add  mask, mask     ; mask ⟸ 1
        jnz  loop           ; 24 passes (mask → 0)
        jmp  OS
a:      dw   1234
b:      dw   567
r:      dw   ?
mask:   dw   ?
t:      dw   ?
```

۳ تقسیم و باقی‌مانده با تفریق‌های متوالی

```
        mov  q, #0          ; x / y   (x ≥ 0, y > 0)
loop:   sub  x, y           ; repeated subtraction
        jneg done
        add  q, #1
        jmp  loop
done:   add  x, y           ; x = remainder
        jmp  OS
x:      dw   100
y:      dw   7
q:      dw   ?
```

زوج/فرد: `and t,#1` ؛ باقی‌ماندهٔ 2^k: `and t,#(2^k-1)` .

۳ شمارش بیت‌های ۱ (popcount)

ماسک ۱، ۲، ۴، ...؛ `and` بیت را جدا می‌کند و پرچم z جواب «صفر بود؟» را می‌دهد. توازن = popcount and 1.

```
        mov  cnt, #0
        mov  mask, #1
loop:   mov  t, x
        and  t, mask        ; z=1 if bit is 0
        jz   skip
        add  cnt, #1
skip:   add  mask, mask     ; next bit
        jnz  loop           ; 24 bits (mask → 0)
        jmp  OS
x:      dw   0B5A3h
cnt:    dw   ?
mask:   dw   ?
t:      dw   ?
```

۱۴۰۲ سؤال ۳

۳ توان ۲ بودن

```
        mov  t, x          ; x & (x-1) == 0 ?
        sub  t, #1
        and  t, x
        jz   yes
        mov  res, #0
        jmp  OS
yes:    mov  res, #1
        jmp  OS
x:      dw   4096
res:    dw   ?
t:      dw   ?
```

۳ قدر مطلق و بیشینهٔ دو عدد

1 + not(x) = -x (اگر `neg` نداریم).

```
        add  x, #0         ; PSR ← x
        jneg negx
        jmp  OS
negx:   xor  x, #FFFFFFh   ; -x = not(x) + 1
        add  x, #1
        jmp  OS
x:      dw   -37
```

```
        mov  mx, a
        mov  t, b
        sub  t, a          ; t = b - a
        jneg done
        mov  mx, b
done:   jmp  OS
a:      dw   -4
b:      dw   9
mx:     dw   ?
t:      dw   ?
```

۸ الگوهای سطح ۴: مرتب‌سازی

۴ Gnome (یک حلقه) با جابه‌جایی XOR

اگر A[i] ≥ A[i+1] جلو برو؛ وگرنه جابه‌جا کن و یکی عقب برگرد (اگر i = 0 نیست). **برای عددهای برابر حتماً «≥» را مرتب حساب کن**، وگرنه حلقهٔ بی‌پایان داریم.

```
        mov  p, #A         ; p → A[i]
loop:   mov  q, p
        add  q, #4         ; q → A[i+1]
        mov  t, (q)
        sub  t, (p)        ; t = A[i+1] - A[i]
        jneg swap          ; out of order?
        add  p, #4         ; in order: i = i + 1
        mov  t, p
        sub  t, #A+396     ; at last element?
        jnz  loop
        jmp  OS
swap:   xor  (p), (q)      ; xor swap A[i], A[i+1]
        xor  (q), (p)
        xor  (p), (q)
        mov  t, p
        sub  t, #A         ; i = 0 ?  then stay
        jz   loop
        sub  p, #4         ; else i = i - 1
        jmp  loop
p:      dw   ?
q:      dw   ?
t:      dw   ?
A:      dw   100 dup(?)
```

اسلاید ماشین ۷ | ۱۳۹۹ سؤال ۲ | ۱۴۰۱ سؤال ۲

۴ حبابی (دو حلقه)

بعد از هر گذر، بزرگ‌ترین عنصر سر جایش می‌نشیند ⇒ تعداد مقایسه‌ها هر گذر یکی کم می‌شود.

```
        mov  pass, #99
outer:  mov  p, #A
        mov  n, pass       ; compare pass pairs
inner:  mov  q, p
        add  q, #4
        mov  t, (q)
        sub  t, (p)        ; A[i+1] - A[i]
        jneg swap
        jmp  next
swap:   xor  (p), (q)
        xor  (q), (p)
        xor  (p), (q)
next:   add  p, #4
        sub  n, #1
        jnz  inner
        sub  pass, #1      ; largest is now in place
        jnz  outer
        jmp  OS
pass:   dw   ?
n:      dw   ?
p:      dw   ?
q:      dw   ?
t:      dw   ?
A:      dw   100 dup(?)
```

تله‌های مرتب‌سازی

- **عددهای تکراری:** اگر فقط «> 0» مرتب حساب شود (مثل `jpos` در ۱۴۰۱)، دو عدد برابر بی‌نهایت جابه‌جا و عقب‌جلو می‌شوند. آرایهٔ `dw 100 dup(0)` دقیقاً همین حالت است!
- **مرز i = 0:** یا شرط «اگر i > 0 عقب برو» بگذار، یا یک **نگهبان** (کوچک‌ترین عدد ممکن) قبل از آرایه (اسلاید ماشین ۷: `80000000h`).
- **سرریز در مقایسه با تفریق:** A[i+1] - A[i] اگر علامت‌ها مخالف و اعداد بزرگ باشند سرریز می‌کند (مثال نگهبان ماشین ۷).
- **گام اشاره‌گر** = طول کلمه بر حسب واحد (۱۴۰۱: ۳، ۹۹ و ۱۴۰۲: ۴، ۹۸: ۸). در ۹۹ `inc4/dec4` دقیقاً برای همین است.
- **آدرس آخرین عنصر** = A + (n-1)·k ؛ ۱۴۰۲: A+297 ⇒ ۱۴۰۱، A+396.

۹ الگوهای سطح ۵: کد خودتغییرده و ترفندهای پیشرفته

۵ پیمایش آرایه با تغییر فیلد آدرس

وقتی آدرس‌دهی غیرمستقیم نداریم: فیلد آدرس دستور را هر دور یک کلمه جلو ببر. موقعیت فیلد را دقیق حساب کن (اینجا addr2 از بیت ۳۰ = واحد ۵ شروع می‌شود). در اجرای دوباره فیلد باید دوباره مقداردهی شود.

```
        mov  n, #100
loop:   add  sum, A        ; addr2 field walks A
        add  loop+5, #4    ; addr2 = units 5..8
        sub  n, #1
        jnz  loop
        jmp  OS
n:      dw   ?
sum:    dw   0
A:      dw   100 dup(?)
```

اسلاید ماشین ۳ | ۱۴۰۰

۵ دستور «چشمک‌زن» با XOR

XOR با یک ثابت، بیت‌های خود دستور را عوض می‌کند و دو بار XOR آن را برمی‌گرداند. اینجا داده بین ۱ و ۰ جابه‌جا می‌شود (۵ = x بعد از ۱۰ دور). در ۱۴۰۱ با ثابت 43Dh کل دستور `add a,#-2` به `jmp next` تبدیل می‌شد.

```
        mov  n, #10
loop:   add  x, #1         ; data: 1,0,1,0,...
        xor  loop+5, #1    ; flip bit 0 of data
        sub  n, #1
        jnz  loop
        jmp  OS
n:      dw   ?
x:      dw   0
```

۱۴۰۱ سؤال ۳

۵ داده = دستور

هیچ تفاوتی بین داده و دستور در حافظه نیست. اگر PC به یک `dw` برسد (یا برنامه از آن شروع شود)، بیت‌هایش رمزگشایی و اجرا می‌شوند.

Written	Executed as
`98: dw -1.0e+109 (F6000000)`	`je R0,R0,330h`
`98: dw 5000000h,1040000h…`	`mov R5,#1 ; mov R4,#8 …`
`98: dw -40.5e+112 (FA220000)`	`fjnan1 R2,R2,out1`
`99: dw 41006h`	`add R1,R0 ; swap (R1),…`
`1400: dw 200Bh,2005h,…`	`add A+8,A ; add A+8,A+4 …`

۵ شرط پایان پنهان

- **سرریز ماسک:** `add m,m` بعد از w دور صفر می‌شود ⇒ `jnz` رد می‌شود (۱۴۰۲).
- **برخورد اشاره‌گرها:** p - q منفی/صفر (۹۹).
- **رسیدن یک فیلد به مقدار هدف:** `jne Arr+5,#B-8,Arr` (۱۴۰۰).
- **شمارنده‌ای که دو بار کم می‌شود:** چند `loop-` پشت‌سرهم (۹۸) ⇒ هنگام خروج دستورهای بعدی هم یک بار اجرا می‌شوند.

۱۰ ممیز شناور در برنامه‌ها

۳ سری تیلور با دقت ε

جملهٔ بعد را از جملهٔ قبل بساز: $t_i = t_{i-1} \cdot x / i$؛ تا وقتی $|t| > \varepsilon$ ادامه بده (دو پرش: $t > \varepsilon$ یا $t < -\varepsilon$). ثابت‌های ممیز شناور را مستقیم با بیت‌هایشان بار کن.

Value	IEEE single	Value	IEEE single
1.0	3F800000	2^{-10}	3A800000
-1.0	BF800000	-2^{-10}	BA800000
2.0	40000000	0.5	3F000000
+∞	7F800000	NaN	7FC00000

برای sin و cos: $t_k = -t_{k-1}\cdot x^2/((2k)(2k+1))$. برای x خیلی منفی بهتر است $e^x = 1/e^{|x|}$.

۳ دسته‌بندی عدد با میدان نما

E	F	Class
0	0	±0
0	≠0	denormalized
1..254	any	normalized
255	0	±∞
255	≠0	NaN

بدون دستور ویژه: نما = (x >> 23) and FFh؛ با `and x,#7F800000h` و مقایسه با 0 یا 7F800000h. دستورهای ۹۸: `fjnan1/2`، `fjden1/2`، `fjinf1/2`. تبدیل `cti` / `ctf` صحیح ⇄ ممیز شناور، `efp` بخش کسری.

عدد اعشاری با نمای دودویی در امتحان ۹۸: `-40.5e+112` یعنی $-40.5 \times 2^{112} = -1.010001 \times 2^{117}$ ⇒ FA220000h ⇒ E = 244.

۱۱ ایده‌های اسلایدها و جزوه (مرور سریع)

ایده	ماشین	نکتهٔ کلیدی	جای حل در جزوه
مجموع ۷ جملهٔ فیبوناچی بدون پرش	۲	باز کردن کامل حلقه؛ t1 و t2 به نوبت جملهٔ جدید را می‌گیرند	مثال ۲-۱۰
جابه‌جایی بدون متغیر کمکی	۲	سه xor	تمرین ۲-۴
مجموع ۱۰۰ جملهٔ فیبوناچی متناوب	۳	دو جمله در هر دور: + و -؛ ۵۰ دور؛ ثابت one	برنامهٔ ۱ فصل ۳
جمع آرایه با غیرمستقیم	۳	شمارندهٔ ۲۰۰ با گام ۲ (یک ثابت کمتر)	برنامهٔ ۲ فصل ۳
جمع آرایه بدون غیرمستقیم	۳	سه اشاره‌گر ثباتی، گام ۴ (واحد نیبلی) (کد خودتغییرده) `add loop+3,two`	برنامهٔ ۳ فصل ۳
جمع دو بردار	۴	سه اشاره‌گر ثباتی، گام ۴ (واحد نیبلی)	ماشین ۴
جمع آرایه با شاخص‌دار	۵	`mov R3,A(R1)` ؛ صفر کردن با xor	ماشین ۵
جمع R0..R62	۵	کد خطی (1F80h+k) یا حلقهٔ خودتغییرده روی فیلد r2	تکلیف ۱ ماشین ۵
۲۰ جملهٔ فیبوناچی در آرایه	تک‌آدرسه	سه اشاره‌گر p1، p2، p3 و آدرس‌دهی غیرمستقیم	ماشین تک‌آدرسه
مجموع فیبوناچی روی پشته	۶	`pop x / sp++` = کپی بالای پشته؛ اتحاد F_{n+1}-1	ماشین ۶
محاسبهٔ عبارت لهستانی	۶	عملوند push، عملگر روی دو عنصر بالا	بخش نمادگذاری لهستانی
مرتب‌سازی Gnome	۷	نگهبان قبل از آرایه؛ پرش نسبی؛ سرریز مقایسه	فصل ۵
جمع ۱۰۰ عدد اول	۷	غربال اراتستن فقط با جمع (ضرب/تقسیم نداریم)؛ جواب 24133	تکلیف فصل ۵
جابه‌جایی برنامه در حافظه	—	آدرس‌های بی‌واسطه و اشاره‌گرهای داده؛ راه‌حل ثبات قطعه	فصل ۵

۱۲ حل کامل میان‌ترم ۱۴۰۲

ماشین: 2^{24} واحد ۶ بیتی، کلمه ۲۴ بیت (۴ واحد)، مستقیم/غیرمستقیم/بی‌واسطه، مکمل دو، PSR = z s (فقط دستورهای حسابی/منطقی).

Format	Fields (bits)	Opcodes
I (54b = 9u)	op3 | addr1 24 | x y z | addr2/data 24	000 mov, 001 and, 010 or, 011 xor, 100 sub, 101 add
II (30b = 5u)	op3 | address 24 | x 1 1	110: x=0 jz, x=1 jnz; 111: x=0 jneg, x=1 jmp
modes	x=1: opr1 = M(M_{a1})	yz: 00 direct, 01 indirect, 10 data, 11 = format II

۱ طول ثبات‌ها

IR = ۵۴، MBR = ۲۴، MAR = PC = ۲۴ (قالب I)، PSR = ۲ بیت.

۲ جمع متناوب پیشوندی

متغیر s مجموع جاری است؛ حلقه دو عنصر در هر دور (j زوج: +، j فرد: -). با ۱۰۰ عدد تصادفی آزموده شد.

```
        org  0
        mov  s, #0          ; running alternating sum
        mov  p, #A
        mov  n, #50
loop:   add  s, (p)         ; j even: s += A[j]
        mov  (p), s         ; A[j] = s
        add  p, #4
        sub  s, (p)         ; j odd:  s -= A[j]
        mov  (p), s
        add  p, #4
        sub  n, #1
        jnz  loop
        jmp  OS_Control_Point
s:      dw   ?
p:      dw   ?
n:      dw   ?
A:      dw   100 dup(?)
```

۳ تحلیل

آدرس‌ها: mov@0، and@9، here@18 (12h)، sub@23، add@32 (20h)، jnz@41، jz@46؛ c = 100h (c+4 = 104h) مقدار ۱)، s = 108h (s+4 = 10Ch).

- `here+14` = 32 = `add c+4,s-4` و `here-18` = 0 = `mov`.
- `s-4` = 104h = **همان c+4** ⇐ `add c+4,s-4` یعنی ماسک = ماسک × ۲.
- بیت k ام c ⇐ `mov s+4,c+4 / and s+4,c` جدا می‌شود؛ اگر صفر بود (z = 1) افزایش رد می‌شود.
- `sub s,#FFFFFFh` = s + 1 = (-1) - s.
- ماسک ۱، ۲، ۴، …، 2^{23}؛ بعد از ۲۴ دور سرریز و صفر ⇒ z = 1 ⇐ `jnz` رد و `jz OS` اجرا می‌شود.

جواب: s = **تعداد بیت‌های ۱ عدد** (popcount) **c**؛ در پایان c+4 = 0 و s+4 موقت است. آزمون: c = 5BE6A3h ⇒ s = 14.

۴ کد ماشین

Addr	Source	Fields (binary)	Octal (2 digits = 1 unit)
0	`mov s+4, c+4`	000 00000000000000100001100 000 00000000000000100000100	00 00 00 41 40 00 00 04 04
9	`and s+4, c`	001 00000000000000100001100 000 00000000000000100000000	10 00 00 41 40 00 00 04 00
12	`here: jz here+14`	110 00000000000000000100000 011	60 00 00 04 03
17	`sub s, #FFFFFFh`	100 00000000000000100001000 010 11111111111111111111111	40 00 00 41 02 77 77 77 77
20	`add c+4, s-4`	101 00000000000000100000100 000 00000000000000100000100	50 00 00 40 40 00 00 04 04
29	`jnz here-18`	110 00000000000000000000000 111	60 00 00 00 07
2E	`jz OS_Control_Point`	110 11111111111111100000000 011	67 77 77 40 03
100	`c: dw ?, 1`	?????????????????????? 00000000000000000000001	?? ?? ?? ?? 00 00 00 01
108	`s: dw 0, ?`	00000000000000000000000 ??????????????????????	00 00 00 00 ?? ?? ?? ??

Symbols: here=12h, c=100h, s=108h

۱۳ حل کامل میان‌ترم ۱۴۰۱

ماشین: 2^{21} واحد ۷ بیتی، کلمه ۲۱ بیت (۳ واحد)، مستقیم/غیرمستقیم/بی‌واسطه، PSR = z s v. (در جدول سؤال، jover با «v = 0» آمده که احتمالاً منظور v = 1 است.)

Format	Fields (bits)	Opcodes
I (49b = 7u)	op4 | i | addr1 21 | xy | addr2/data 21	0000 mov, 0001 and, 0010 or, 0011 xor, 0100 cmp, 0101 sub, 0110 add
II (28b = 4u)	op4 | i | addr1 21 | 1 1	0000 clr, 0001 inc, 0010 dec, 0011 not, 0100 neg, 0101 call ($M_{1FFFFD} \leftarrow$ PC), 0110 jmp, 0111 jz, 1000 jnz, 1001 jneg, 1010 jpos (svz=0), 1011 jover
modes	i=1: opr1 / target indirect	xy: 00 direct, 01 indirect, 10 data, 11 = format II

۱ طول ثبات‌ها

IR = ۴۹، PSR = ۳ MAR = PC = ۲۱، MBR = ۲۱ بیت.

۲ تکمیل برنامه (Gnome با XOR)

جاهای خالی: `ptr1`، `cmp (ptr2),(ptr1)`، `xor (ptr2),(ptr1)`، `sub ptr1,#2`، `xor (ptr1),(ptr2)` (همراه dec یعنی ۳ واحد عقب)، `cmp ptr1,#A+297`، `jnz loop1`.

```
        org  100h
loop1:  mov  ptr2, ptr1      ; ptr2 = ptr1
        add  ptr2, #3        ; next word (3 units)
loop2:  cmp  (ptr2), (ptr1)  ; A[i+1] - A[i]
        jpos cont            ; > 0 : in order
        xor  (ptr1), (ptr2)  ; xor swap
        xor  (ptr2), (ptr1)
        xor  (ptr1), (ptr2)
        dec  ptr1            ; ptr1 -= 3
        sub  ptr1, #2
        jmp  loop1
cont:   add  ptr1, #3
        cmp  ptr1, #A+297    ; at A[99]? (99x3)
        jnz  loop1
        jmp  (1FFFFDh)
A:      dw   100 dup(0)
ptr1:   dw   A
ptr2:   dw   ?
```

دو ایراد قالب داده‌شده (با شبیه‌ساز دیده شد)

- `jpos` فقط برای «> 0» می‌پرد ⇒ اگر دو عنصر برابر باشند، برنامه بی‌پایان عقب‌وجلو می‌رود. آرایهٔ `dw 100 dup(0)` خودش همین حالت است.
- در i = 0 بعد از جابه‌جایی، ptr1 به A-3 می‌رود. آن کلمه، سه واحد آخر دستور `jmp (1FFFFDh)` است که به عنوان عدد برابر **-9** خوانده می‌شود؛ پس برنامه فقط برای مقادیر بزرگ‌تر از -9 درست کار می‌کند.

نسخهٔ امن (فقط وارونگی واقعی جابه‌جا می‌شود و در i = 0 می‌ماند؛ با عددهای تکراری آزموده شد):

```
        org  100h
loop1:  mov  ptr2, ptr1
        add  ptr2, #3
        cmp  (ptr2), (ptr1)  ; A[i+1] - A[i]
        jneg swap            ; real inversion only
        add  ptr1, #3
        cmp  ptr1, #A+297
        jnz  loop1
        jmp  (1FFFFDh)
swap:   xor  (ptr1), (ptr2)
        xor  (ptr2), (ptr1)
        xor  (ptr1), (ptr2)
        cmp  ptr1, #A        ; at A[0]: stay
        jz   loop1
        sub  ptr1, #3
        jmp  loop1
A:      dw   100 dup(0)
ptr1:   dw   A
ptr2:   dw   ?
```

۳ تحلیل

آدرس‌ها: org 1: xor@1، loop@8، next@15 (Fh)، dec@22، jnz@26، jmp@30؛ a = 100h، c = 103h، arr = 106h، k = 232h (arr و k بی‌استفاده‌اند).

- `c-3` = 0 ⇒ `xor a,a` ⇒ a = 0 = 100h.
- `loop+1` = واحد ۹: کلمهٔ واحدهای ۹..۱۱ = بیت‌های ۷..۲۷ دستور `add a,#-2` = ۱۹ بیت پایین addr1 + xy = 402h.
- addr1 = 0Fh (= next) ⇒ 402h ⊕ 43Dh = 03Fh و xy = 11 ⇒ دستور با همان کد عمل 0110 در قالب II یعنی `jmp next`!
- هر دور یک بار XOR می‌شود ⇒ دورهای ۱، ۳، ۵، ۷، ۹: `add a,#-2`؛ دورهای زوج: پرش. c از ۱۰ تا ۰.

جواب: c = 0، a = **-10** (= 1FFFF6h) و دستور در پایان به حالت اول برگشته است (تعداد XORها زوج است). **در صورت سؤال قبل از 43Dh علامت # نیامده؛ اگر مستقیم خوانده شود، M[43Dh] نامعلوم است (با صفر، چیزی عوض نمی‌شود و a = -20).**

۴ کد ماشین (مطابق متن چاپ‌شده، `43Dh` مستقیم: xy = 00؛ نسخهٔ بی‌واسطه xy = 10 است)

Addr	Source	Fields (binary)	Hex per 7-bit unit
1	`xor a, c-3`	0011 0 0000000000100000000 00 0000000000100000000	18 00 08 00 00 02 00
8	`loop: add a, #-2`	0110 0 0000000000100000000 10 1111111111111111110	30 00 08 02 7F 7F 7E
F	`next: xor loop+1, 43Dh`	0011 0 0000000000000001001 00 0000000010000111101	18 00 00 24 00 08 3D
16	`dec c`	0010 0 0000000000100000011 11	10 00 08 0F
1A	`jnz loop`	1000 0 0000000000000001000 11	40 00 00 23
1E	`jmp (1FFFFDh)`	0110 1 1111111111111111101 11	37 7F 7F 77
100	`a: dw 123456h`	1001000011010001010110	48 68 56
103	`c: dw 10`	0000000000000001010	00 00 0A
106	`arr: dw 100 dup(?)`	????????????????????? ????????????????????? ????????????????????? …	?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? …
232	`k: dw 13`	0000000000000001101	00 00 0D

Symbols: loop=8h, next=Fh, a=100h, c=103h, arr=106h, k=232h

۱۴ حل کامل میان‌ترم ۱۴۰۰

ماشین: 2^{14} کلمهٔ ۱۶ بیتی با واحد ۴ بیتی (2^{16} واحد)، مستقیم/غیرمستقیم/بی‌واسطه، مکمل دو، بدون ثبات و پرچم.

Format	Fields (bits)	Opcodes
I (36b = 9u)	op3 | addr1 16 | d | addr2/data 16	000 mov, 001 add, 010 sub (d=0: opr2 = (M_{a2}), d=1: data)
II (20b = 5u)	op3 | addr1 16 | x	011: x=0 inc, x=1 dec; 100: x=0 jmp (addr1), x=1 call (M_{FFFE} ← PC)
III (52b = 13u)	op3 | addr1 16 | d | addr2/data 16 | addr3 16	101 je, 110 jne, 111 jl: if (M_{a1}) ? opr2 then PC ← addr3

۱ طول ثبات‌ها

IR = ۵۲، MBR = ۱۶، MAR = PC = ۱۶ (قالب III).

۲ مجموع ۱۰۰ جملهٔ فیبوناچی

دو جمله در هر دور، ۴۹ دور؛ جواب به پیمانهٔ 2^{16} (سرریز). راه کوتاه‌تر: فقط F_{101} را بساز و یکی کم کن.

```
        org  0
        mov  a, #0          ; F0
        mov  b, #1          ; F1
        mov  sum, #1        ; F0 + F1
        mov  n, #49         ; 2 terms per pass
loop:   add  a, b           ; a = F2, F4, ...
        add  sum, a
        add  b, a           ; b = F3, F5, ...
        add  sum, b
        dec  n
        jne  n, #0, loop
        jmp  (FFFEh)        ; return address
a:      dw   ?
b:      dw   ?
sum:    dw   ?
n:      dw   ?
```

۳ تحلیل

۹ کلمهٔ Arr در واقع **۴ دستور** است (هر کدام ۹ واحد):

`add A+8, A`	`add A+8, A+4`
`add Arr+1, #8`	`add Arr+5, #4`

- `mov B-396, A+400` : B = A+400 (۱۰۰ کلمه × ۴)، پس M[B] = 1 ← A[1].
- دو دستور اول: A[i] += A[i+2] و **A[i+1] = A[i] + A[i+2]** ⇒ A[i+2] += A[i+1].
- `add Arr+1,#8` : کلمهٔ Arr+1 = بیت‌های ۴..۱۹ دستور اول و addr1 یک بیت جابه‌جاست ⇒ ۸+ یعنی ۴ + addr1. `Arr+5` = دقیقاً addr2 دستور اول. `Arr+10` و `Arr+14` همین کار را برای دستور دوم می‌کنند.
- `jne Arr+5,#B-8,Arr` : تا وقتی addr2 دستور اول به B-8 = A[98] نرسیده، تکرار ⇒ ۹۸ دور.

جواب: A[0..99] را با **۱۰۰ جملهٔ اول فیبوناچی** پر می‌کند (به پیمانهٔ 2^{16}) و با `jmp (FFFEh)` برمی‌گردد.

نکته: عددهای Arr برای org 0 ساخته شده‌اند (A = 51h، Arr = 9). با org 100h چاپ‌شده، مقادیر درست 21Dh، 82h، 2150h، 5520h، B201h، 1202h، 2015h، 202Bh و 4 می‌شد. با شبیه‌ساز: org 0 ⇐ فیبوناچی کامل؛ org 100h ⇐ حلقهٔ بی‌پایان.

۴ کد ماشین (مطابق متن، org 100h)

Addr	Source	Machine code (hex, by field)
100	mov B-396, A+400	000 0000000101010101 0 0000001011100001
109	Arr: dw 200Bh, 2005h, 1200h	200B 2005 1200
115	dw B200h, 5520h, 150h	B200 5520 0150
121	dw 130, 29, 4	0082 001D 0004
12D	add Arr+10, #8	001 0000000100010011 1 0000000000001000
136	add Arr+14, #4	001 0000000100010111 1 0000000000000100
13F	jne Arr+5, #B-8, Arr	110 0000000100001110 1 0000001011011001 0000000100001001
14C	jmp (FFFEh)	100 1111111111111110 0
151	A: dw 100 dup(0)	0000 0000 0000 0000 0000 0000 …
2E1	B: dw 1	0001

Symbols: Arr=109h, A=151h, B=2E1h

۱۵ حل کامل میان‌ترم ۱۳۹۹

ماشین: 2^{24} واحد ۶ بیتی، کلمه ۲۴ بیت (۴ واحد)، R0..R31، آدرس بازگشت OS در R31، ثابتی و حافظه‌ای (مستقیم و غیرمستقیم) و بی‌واسطه، مکمل دو.

Format	Fields (bits)	Opcodes
I (18b = 3u)	00 op4 | i1 r1(5) | i2 r2(5) / qdata 6	mov, add, sub, and, or, xor, swap, addq, subq, movq (0000..1001)
II (12b = 2u)	010 op4 | r 5	jmp r (PC ← (r)), inc, dec, clr, not, neg, inc4, dec4
III (36b = 6u)	011 op4 | r 5 | addr 24	mov r,addr / addr,r / r,(addr) / (addr),r, jnz, jz, jneg (<0), jpos (≥0), loop, jmp+, call
V (30b = 5u)	1 | r 5 | data 24	mov r,#data

۱ طول ثبات‌ها

IR = ۳۶، MBR = ۲۴، MAR = PC = ۲۴ (قالب III)، R0..R31 هر کدام ۲۴ بیت.

۲ مرتب‌سازی صعودی ۱۰۰ کلمه

Gnome با دستور `swap` و گام `inc4/dec4` . `jpos` اینجا ≥ 0 است، پس عددهای برابر مشکلی ندارند. با عددهای تکراری آزموده شد.

```
        org  0
        mov  R1,#array       ; p → A[i]
        mov  R5,#array+396   ; → A[99] (99x4)
        mov  R6,#array
loop:   mov  R2,R1
        inc4 R2              ; q → A[i+1]
        mov  R3,(R2)
        sub  R3,(R1)         ; A[i+1] - A[i]
        jpos R3,next         ; ⩾ 0 : in order
        swap (R1),(R2)       ; one instruction swap
        mov  R3,R1
        sub  R3,R6
        jz   R3,loop         ; i = 0 : stay
        dec4 R1              ; i = i - 1
        jnz  R6,loop         ; R6 ≠ 0: always
next:   inc4 R1
        mov  R3,R1
        sub  R3,R5
        jneg R3,loop         ; p < last
        jmp  R31             ; back to the OS
array:  dw   100 dup(?)
```

۳ تحلیل: دستورهایی که واقعاً اجرا می‌شوند

addr	executed	note
0	mov R1,#100	R1 = 100 units
5	mov R0,#array	R0 = 24h = 36
10	add R1,R0	from dw 41006h
13	swap (R1),(R0)	loop-6 lands here
16	addq R0,4	bits of 'mov R1,#807004h'
19	subq R1,4	loop:
22	mov R2,R0	
25	sub R2,R1	
28	jneg R2,13	while R0 < R1
34	jmp R31	back to OS

بیت‌ها:

`000001 000001 000000 | 000110 100001 100000 | 000111 000000 000100`

⇒ add R1,R0 | swap (R1),(R0) | addq R0,4 با i1 = i2 = 1.

جواب: R0 از اول آرایه و R1 = array + 100 واحد (= کلمهٔ ۲۵) به سمت هم حرکت می‌کنند و عنصرها را جابه‌جا می‌کنند ⇒ **ترتیب ۲۶ کلمهٔ اول آرایه (A[0..25]) وارونه می‌شود.** بقیهٔ آرایه دست‌نخورده می‌ماند. (تله: #100 بر حسب واحد است، نه کلمه.)

۴ کد ماشین

Addr	Source	Fields (binary)	Octal (2 digits = 1 unit)
0	mov R1,#100	1 00001 00000000000000000100100	41 00 00 01 44
5	mov R0,#array	1 00000 00000000000000000100100	40 00 00 00 44
A	dw 41006h	00000100000100000000110	01 01 00 06
E	mov R1,#807004h	1 00001 10000000011100000000100	41 40 07 00 04
13	loop: subq R1,4	00 1000 0 00001 000100	10 01 04
16	mov R2,R0	00 0000 0 00010 0 00000	00 02 00
19	sub R2,R1	00 0010 0 00010 0 00001	02 02 01
1C	jneg R2,loop-6	011 0110 00010 00000000000000000001101	33 02 00 00 00 15
22	jmp R31	010 0000 11111	20 37
24	array: dw 100 dup(?)	???????????????????????? ???????????????????????? ???????????????????????? …	?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? …

Symbols: loop=13h, array=24h

۱۶ حل کامل میان‌ترم ۱۳۹۸

ماشین: 2^{29} کلمهٔ ۳۲ بیتی با واحد ۴ بیتی (2^{32} واحد)، R0..R15 (**R0 همیشه صفر**)، ثابتی، بی‌واسطه و شاخص‌دار، مکمل دو و IEEE 754 تک‌دقتی.

Format	Fields (bits)	Opcodes
I (40b = 10u)	0000 | r | data 32	mov r,#data
II (12b = 3u)	op4 | r1 | r2	0001 mov … 0110 xor, 0111 ctf, 1000 cti, 1001 efp, 1010 fadd, 1011 fsub, 1100 fmul, 1101 fdiv, 1110 fsqrt
III (48b = 12u)	1111 op4 | r1 | r2 | addr 32	0000 mov r1,addr(r2), 0001 mov addr(r2),r1, 0010 loop-, 0011 loop+, jl, jh, je, jne, fjl, fjh, fjnan1/2, fjden1/2, fjinf1/2

۱ طول ثبات‌ها

IR = ۴۸، MBR = ۳۲، MAR = PC = ۳۲ (قالب III)، R0..R15 هر کدام ۳۲ بیت.

۲ e^x با دقت 2^{-10}

ترفندها: `x(R0)` با R0 = 0 یعنی آدرس‌دهی مستقیم؛ ثابت‌ها با بیت‌های IEEE بار می‌شوند؛ شمارندهٔ i خودش ممیز شناور است (یا با `ctf`). برای x = -6 تا 7 آزموده شد (بیشترین خطا حدود 1.4×10^{-4}).

```
        org  0
        mov  R1,x(R0)        ; R0=0 → direct
        mov  R2,#3F800000h   ; term = 1.0
        mov  R3,R2           ; sum  = 1.0
        mov  R4,R2           ; i    = 1.0
        mov  R5,R2           ; 1.0
        mov  R6,#3A800000h   ;  2^-10
        mov  R7,#BA800000h   ; -2^-10
loop:   fmul R2,R1           ; term = term * x
        fdiv R2,R4           ;        / i
        fadd R3,R2           ; sum += term
        fadd R4,R5           ; i += 1
        fjh  R2,R6,loop      ; term >  2^-10
        fjl  R2,R7,loop      ; term < -2^-10
        mov  ex(R0),R3       ; store e^x
        je   R0,R0,OS
x:      dw   ?
ex:     dw   ?
```

۳ تحلیل: اجرا از org 0 یعنی از روی «داده»

addr	executed	from
0	`je R0,R0,330h`	float1 + int1
330	`mov R5,#1`	dw 5000000h…
33A	`mov R4,#8`	
344	`mov R1,#320h`	
34E	`mov R10,#0`	
358	`mov R11,R0`	R11 = 0
35B	`mov R2,8(R1)`	loop: array-8
367	`fjnan1 R2,R2,38Bh`	-40.5e+112, 38BF…
373	`fjden1 R2,R2,39Ah`	…C22h, 39Ah
37F	`loop- R1,R4,loop`	R1 -= 8
38B	`add R10,R5`	out1: NaN
39A	`add R11,R5`	out2: denormal

جواب: آرایه را از A[99] تا A[1] پیمایش می‌کند؛ **R10 = تعداد NaNها** و **R11 = تعداد اعداد غیرنرمال**. جزئیات: A[0] بررسی نمی‌شود (حلقه وقتی R1 ≤ 8 تمام می‌شود)، و هنگام خروج دستورهای بعدی هم یک بار اجرا می‌شوند: اگر A[1] عادی باشد R10 و R11 هر کدام +۱، اگر NaN باشد فقط R11 +۱، اگر غیرنرمال باشد هیچ.

۴ کد ماشین (هگز، هر رقم یک واحد)

Addr	Source	Machine code (hex, by field)
0	`float1: dw -1.0e+109`	F6000000
8	`int1: dw 3300000h`	03300000
10	`array: dw 100 dup(?)`	???????? ???????? ???????? ???????? ???????? ???????? …
330	`dw 5000000h,1040000h`	05000000 01040000
340	`dw 80100h,3200Ah,0`	00080100 0003200A 00000000
358	`mov R11,R0`	1 B 0
35B	`loop: mov R2,array-8(R1)`	F0 2 1 00000008
367	`dw -40.5e+112`	FA220000
36F	`dw 38BFC22h,39Ah`	038BFC22 0000039A
37F	`loop- R1,R4,loop`	F2 1 4 0000035B
38B	`out1: add R10,R5`	2 A 5
38E	`loop- R1,R4,loop`	F2 1 4 0000035B
39A	`out2: add R11,R5`	2 B 5
39D	`loop- R1,R4,loop`	F2 1 4 0000035B
3A9	`je R0,R0,OS_control_point`	F6 0 0 FFFF0000

Symbols: float1=0h, int1=8h, array=10h, loop=35Bh, out1=38Bh, out2=39Ah